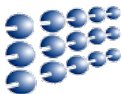




Facultatea
de Automatică
și Calculatoare
Universitatea Tehnică
din Cluj-Napoca



Fundamental Programming Techniques

Assignment 3 - Support Presentation (Part 3)

Computer Science Department, TUCN

Spring 2026

Lambda Expressions

- **Anonymous block of code**

- ▶ Describes an anonymous function that has no name, no return type, no throws clause and no generics

- **Syntax** (<LambdaParametersList>)-> { <LambdaBody> }

- ▶ (<LambdaParametersList>) - a comma-separated list of formal parameters enclosed in parentheses
- ▶ -> - the arrow token
- ▶ { <LambdaBody> } - consists of a single expression or a statement block
 - ★ May declare local variables
 - ★ May use statements including break, continue and return
 - ★ May throw exceptions
- ▶ Classification: implicit typed and explicit typed lambda expressions

Lambda Expressions and Functional Interfaces

- Lambda expressions have different types in different contexts => are poly expressions
- Lambda expression type is **Functional Interface**, the exact type demands on the context in which it is used
 $T \ t = \langle \text{LambdaExpression} \rangle \rightarrow$ The target type of the λ ex is T

Inferring rules used by compiler(they are close related to the abstract method of the Functional Interface)

- T must be a Functional Interface type
- λ ex has the same number and type of parameters as the abstract method of T
- For an implicit λ ex, parameters types are inferred from the abstract method of T
- The type of the returned value from the body of the λ ex should be assignment compatible to the return type of the abstract method of T
- If the body of the λ ex throws any checked exceptions, they must be compatible with the declared throws clause of the abstract method of T
- It is a compile-time error to throw checked exceptions from the body of a λ ex, if its target type's method does not contain a throws clause

```
1  @FunctionalInterface
2  public interface Adder {
3      double add(double n1, double n2);
4  }
5  Added adder = (x, y) -> x + y;
6  double sum1 = adder.add(10.34, 89.11);
```

Lambda Expressions and Functional Interfaces

Common functional interfaces defined in *java.util.function*

Interface Name	Method	Description
<code>Function<T, R></code>	<code>R apply(T t)</code>	Represents a function that takes an argument of type <code>T</code> and returns a result of type <code>R</code>
<code>BiFunction<T, U, R></code>	<code>R apply(T t, U u)</code>	Represents a function that takes two arguments of types <code>T</code> and <code>U</code> , and returns a result of type <code>R</code>
<code>Predicate<T></code>	<code>boolean test(T t)</code>	In mathematics, a predicate is a boolean-valued function that takes an argument and returns true or false.
<code>BiPredicate<T, U></code>	<code>boolean test(T t, U u)</code>	Represents a predicate with two arguments.
<code>Consumer<T></code>	<code>void accept(T t)</code>	Represents an operation that takes an argument, produces side effects, and returns no result.
<code>BiConsumer<T, U></code>	<code>void accept(T t, U u)</code>	Represents an operation that takes two arguments, produces side effects, and returns no result.
<code>Supplier<T></code>	<code>T get()</code>	Represents a supplier that returns a value.
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	Inherits from <i>Function<T, T></i> . Takes an argument and returns a result of the same type.
<code>BinaryOperator<T></code>	<code>T apply(T t1, T t2)</code>	Inherits from <i>BiFunction<T, T, T></i> . Takes two arguments and returns a result of the same type.

```
1 // Example using Function
2 Function<Long, Long> square = x -> x * x;
3 Function<Long, Long> addOne = x -> x + 1;
4 Function<Long, Long> squareAddOne = square.andThen(addOne);
5 System.out.println(squareAddOne.apply(5L));
```

```
1 // Example using Predicate
2 Predicate<Integer> greaterThanTen = x -> x > 10;
3 Predicate<Integer> lessThanOrEqualToTen = greaterThanTen.negate();
4 System.out.println(greaterThanTen.test(10));
5 System.out.println(lessThanOrEqualToTen.test(10));
```

Lambda Expressions and Functional Interfaces

- **Method references**

- ▶ compact, easy-to-read lambda expressions for methods that already have a name

- **Syntax** `<Qualifier>::<MethodName>`

- ▶ `<Qualifier>` depends on the type of the method reference
- ▶ `<MethodName>` is the name of the method

Syntax	Description
<code>TypeName::staticMethod</code>	A method reference to a static method of a class, an interface, or an enum
<code>objectRef::instanceMethod</code>	A method reference to an instance method of the specified object
<code>ClassName::instanceMethod</code>	A method reference to an instance method of an arbitrary object of the specified class
<code>TypeName.super::instanceMethod</code>	A method reference to an instance method of the supertype of a particular object
<code>ClassName:new</code>	A constructor reference to the constructor of the specified class
<code>ArrayTypenew</code>	An array constructor reference to the constructor of the specified array type

```
1 public interface MyPrinter {
2     public void print(String s);
3 }
4
5 // Using lambda expressions
6 MyPrinter myPrinter = s -> System.out.println(s);
7 // Using method references
8 MyPrinter myPrinter = System.out::println;
```

Lambda Expressions and Functional Interfaces

- **Method references - Comparing Objects**

- ▶ Methods of the Comparator interface

```
static <T, U extends Comparable<? super U>> Comparator<T> comparing (Function<? super T, ? extends U> keyExtractor)  
default <U extends Comparable<? super U>> Comparator <T> thenComparing (Function<? super T, ? extends U> keyExtractor)
```

- ▶ Examples - create a *Comparator<Person>* that sorts Person objects based on their last names and first names

```
1 Comparator<Person> lastFirstComp = Comparator.comparing(Person::getLastName).thenComparing(Person::getFirstName);
```

Streams

- Definition

- ▶ a **sequence of elements** from a **source** that supports **aggregate operations**
- ▶ **Sequence of elements**: A stream provides an interface to a sequenced set of values of a specific element type. Streams do not actually store elements. They are computed on demand.
- ▶ **Source**: Streams consume from a data-providing source such as collections, arrays, or I/O resources.
- ▶ **Aggregate operations**: Streams support SQL-like operations and common operations from functional programming languages (e.g., filter, map, reduce, find, match, sorted, etc.)

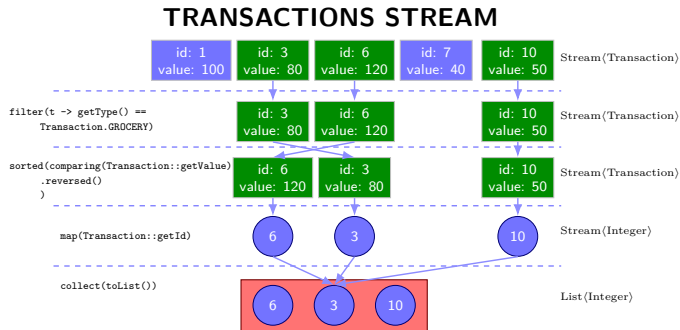
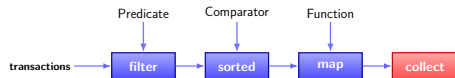
- Features

- ▶ **Pipelining**: Many stream operations return a stream themselves => allows operations to be chained to form a larger pipeline
- ▶ **Internal iteration**: In contrast to collections, which are iterated explicitly (external iteration), stream operations do the iteration behind the scenes for you

Streams

- Example: Find all transactions of type grocery and return a list of transaction IDs sorted in decreasing order of transaction value

```
1 List<Integer> transactionIds = transactions.stream()  
2   .filter(t -> t.getType() == Transaction.GROCERY)  
3   .sorted(comparing(Transaction::getValue).reversed())  
4   .map(Transaction::getId)  
5   .collect(toList());
```



Streams

- Examples - Create a stream from files

```
1  public class Product {
2      private String name;
3      public Product(String name) {
4          this.name = name;
5      }
6      public String getName() {
7          return name;
8      }
9      public void setName(String name) {
10         this.name = name;
11     }
12 }
13
14 public class StreamProcessing {
15     public static void main(String[] args) throws IOException {
16         Stream<String> stream = Files.lines(Paths.get("products.txt"));
17         List<Product> productList = stream.map(line -> new Product(line))
18             .collect(Collectors.toList());
19         productList.stream()
20             .map(Product::getName)
21             .forEach(System.out::println);
22     }
23 }
24 /*
25  * products.txt
26  * apple
27  * juice
28  * bread
29  */
```

Streams

- **Collectors Class** (java.util.stream package) implements various useful reduction operations

Modifier and Type	Method	Description
static <T> Collector<T,?,Double>	averagingDouble(ToDoubleFunction<? super T> mapper)	Returns a Collector that produces the arithmetic mean of a double-valued function applied to the input elements.
static <T> Collector<T,?,Double>	averagingInt(ToIntFunction<? super T> mapper)	Returns a Collector that produces the arithmetic mean of an integer-valued function applied to the input elements.
static <T> Collector<T,?,Double>	averagingLong(ToLongFunction<? super T> mapper)	Returns a Collector that produces the arithmetic mean of a long-valued function applied to the input elements.
static <T,A,R,RR> Collector<T,A,RR>	collectingAndThen(Collector<T,A,R> downstream, Function<R,RR> finisher)	Adapts a Collector to perform an additional finishing transformation.
static <T> Collector<T,?,Long>	counting()	Returns a Collector accepting elements of type T that counts the number of input elements.
static <T,K> Collector<T,?,Map<K,List<T>>>	groupingBy(Function<? super T,? extends K> classifier)	Returns a Collector implementing a "group by" operation on input elements of type T, grouping elements according to a classification function, and returning the results in a Map.

```
1 // Accumulate names into a List
2 List<String> list = people.stream().map(Person::getName).collect(Collectors.toList());
3 // Convert elements to strings and concatenate them, separated by commas
4 String joined = things.stream().map(Object::toString).collect(Collectors.joining(","));
5 // Compute the total salary of all employees
6 int total = employees.stream().collect(Collectors.summingInt(Employee::getSalary));
7 // Group employees by department
8 Map<Department, List<Employee>> byDept = employees.stream().collect(Collectors.groupingBy(Employee
    ::getDepartment));
9 // Compute sum of salaries by department
10 Map<Department, Integer> totalByDept = employees.stream().collect(Collectors.groupingBy(Employee::
    getDepartment, Collectors.summingInt(Employee::getSalary)));
```